Interface Segregation Principle (ISP) - SOLID in Java

1. What is ISP?

The Interface Segregation Principle is the "I" in SOLID.

Rule: A client should NEVER be forced to implement an interface it does not use.

No code should be forced to depend on methods it does not use.

In plain English: Keep interfaces small and focused. Do NOT create fat interfaces.

2. The Problem Without ISP

Use Case: Workplace Management System

A single fat Worker interface forces RobotWorker to implement eat(), attendMeeting() etc.

```
// FAT interface -- violates ISP
public interface Worker {
    void work();
    void eat();            // Robots don't eat!
    void attendMeeting();  // Robots don't attend meetings!
    void writeCode();
    void manageTeam();     // Not every worker manages!
}

// Human developer -- fine, uses everything
public class HumanDeveloper implements Worker {
    public void work()          { System.out.println("Writing features..."); }
    public void eat()           { System.out.println("Lunch break!"); }
    public void attendMeeting() { System.out.println("In standup..."); }
    public void writeCode()     { System.out.println("Coding..."); }
    public void manageTeam()    { System.out.println("Managing team..."); }
}

// Robot worker -- FORCED to implement methods it doesn't need (WRONG)
public class RobotWorker implements Worker {
    public void work()          { System.out.println("Executing task..."); }
    public void eat()           { throw new UnsupportedOperationException("Robots don't eat!"); }
    public void attendMeeting() { throw new UnsupportedOperationException("Robots don't meet!"); }
    public void writeCode()     { System.out.println("Auto-generating code..."); }
    public void manageTeam()    { throw new UnsupportedOperationException("Robots don't manage!"); }
}
```

Problems with this approach

Problem	Impact
RobotWorker throws UnsupportedOperationException	Runtime crashes -- fragile system
Interface polluted with irrelevant methods	Change in eat() forces recompile of RobotWorker
Violates Liskov Substitution too	Cannot safely substitute RobotWorker for Worker
Hard to test	Mocking forces stubbing methods you don't care about

Interface Segregation Principle (ISP) - SOLID in Java

3. The Fix -- Apply ISP

Split the fat interface into small, role-specific interfaces.

Step 1: Define segregated interfaces

```
// Each interface has ONE focused responsibility
public interface Workable   { void work(); }
public interface Eatable    { void eat(); }
public interface Meetable   { void attendMeeting(); }
public interface Codeable   { void writeCode(); }
public interface Manageable { void manageTeam(); }
```

Step 2: Each class implements only what it needs

```
// Human developer -- only relevant interfaces
public class HumanDeveloper implements Workable, Eatable, Meetable, Codeable {
    @Override public void work()          { System.out.println("Working on ticket..."); }
    @Override public void eat()           { System.out.println("Grabbing lunch..."); }
    @Override public void attendMeeting() { System.out.println("In sprint planning..."); }
    @Override public void writeCode()     { System.out.println("Writing clean code..."); }
}

// Robot -- only what makes sense for a robot (no eat, no meeting!)
public class RobotWorker implements Workable, Codeable {
    @Override public void work()      { System.out.println("Executing automated task..."); }
    @Override public void writeCode() { System.out.println("Auto-generating boilerplate..."); }
}

// Tech Lead -- has all capabilities including managing
public class TechLead implements Workable, Eatable, Meetable, Codeable, Manageable {
    @Override public void work()          { System.out.println("Working on architecture..."); }
    @Override public void eat()           { System.out.println("Team lunch..."); }
    @Override public void attendMeeting() { System.out.println("Leading standup..."); }
    @Override public void writeCode()     { System.out.println("Code reviewing..."); }
    @Override public void manageTeam()    { System.out.println("1-on-1s and mentoring..."); }
}
```

Step 3: Consumers depend only on what they need

```
// Sprint runner only cares about Workable -- works for Human, Robot, TechLead
public class SprintRunner {
    private final List<Workable> workers;
    public SprintRunner(List<Workable> workers) { this.workers = workers; }
    public void runSprint() { workers.forEach(Workable::work); }
}

// Lunch scheduler only cares about Eatable -- Humans and TechLead only, not Robot
public class LunchScheduler {
    private final List<Eatable> attendees;
    public LunchScheduler(List<Eatable> attendees) { this.attendees = attendees; }
    public void scheduleLunch() { attendees.forEach(Eatable::eat); }
}
```

Interface Segregation Principle (ISP) - SOLID in Java

4. Unit Testing Becomes Clean

```
@Test
public void testSprintRunner_withRobot() {
    Workable robot = Mockito.mock(Workable.class); // Only mock what is needed
    SprintRunner runner = new SprintRunner(List.of(robot));

    runner.runSprint();

    Mockito.verify(robot).work();
    // No need to stub eat(), attendMeeting() etc. -- they don't exist here!
}
```

5. Real-World Java Example -- java.util itself!

```
// Java's own collections API is a great ISP example:
List<String>       list    = new ArrayList<>();  // Iterable + Collection + List
Iterable<String>   iter    = new ArrayList<>();  // just iteration
Comparable<String> compare = "hello";            // just comparison

// You don't get forced to use sort() just because you needed iterator()
```

6. Before vs After Comparison

Before ISP	After ISP
One fat Worker interface with 5 methods	5 small focused interfaces
RobotWorker throws UnsupportedOperationException	RobotWorker only implements what it needs
Change in eat() triggers recompile everywhere	Change in eat() only affects Eatable implementors
Mocking requires stubbing unused methods	Mock only the interface you care about
Tightly coupled consumers	Consumers depend on minimal surface area

7. Key Insight

ISP and DIP work hand-in-hand.

DIP says: depend on abstractions.

ISP says: keep those abstractions lean.

Together they give you a system that is easy to extend, test, and maintain without cascading changes.

This is exactly how Spring interfaces like ApplicationEventListener, BeanPostProcessor, and InitializingBean are designed -- each one is laser-focused.